
用户信息管理平台

安全漏洞挖掘与修复报告

`http://10.133.25.61:5000`

项目版本: V10.0 - 命令注入漏洞专题

报告日期: 2026年7月16日

今日课程: Day8:命令注入漏洞深度解析

技术栈: Python Flask / subprocess / shell=True / f-string拼接

一、课程背景

命令注入（Command Injection）是一种通过应用程序执行任意系统命令的攻击方式。当应用程序将用户输入拼接到系统命令中，且未做任何过滤时，攻击者可以利用 shell 元字符（如 ;、|、&& 等）注入额外的命令。命令注入漏洞通常被评为高危或严重，因为它直接导致攻击者获得服务器的操作系统控制权。

1.1 命令注入 vs SQL注入

对比项	命令注入	SQL注入
攻击目标	操作系统	数据库
拼接方式	f"ping -c 3 {ip}"	f"SELECT * FROM users WHERE id={id}"
分号;	可分隔多条命令	可分隔多条SQL语句
危害	执行系统命令、控制服务器	窃取/篡改数据库数据
修复方式	避免shell=True，使用参数化	使用参数化查询

1.2 本次新增功能

功能	路由	参数	说明
Ping测试	/ping	ip	用户输入IP，服务端执行 ping -c 3 {ip}

二、漏洞代码分析

2.1 漏洞代码

```
@app.route("/ping", methods=["POST"])
@login_required
def ping():
    ip = request.form.get("ip", "")

    # 【漏洞1】f-string拼接系统命令
    command = f"ping -c 3 {ip}"

    # 【漏洞2】shell=True 启用shell解析
    # 攻击者可利用 ; | && 等注入额外命令
    output = subprocess.check_output(
        command, shell=True, timeout=30
    )
    return render_template("ping.html", output=output)
```

2.2 漏洞成因分析

#	漏洞	说明	攻击利用
1	f-string拼接	用户输入直接拼入命令字符串，未做任何校验	ip=8.8.8.8;id → 执行ping + id命令
2	shell=True	启用shell解析，支持管道、重定向等操作	ip=127.0.0.1 whoami → 执行whoami命令
3	无输入校验	IP地址格式、特殊字符均未过滤	ip=127.0.0.1;cat /etc/passwd → 读取系统文件

三、攻击复现（POC）

3.1 POC 1: 正常Ping

```
请求: POST /ping ip=8.8.8.8
命令: ping -c 3 8.8.8.8
结果: 正常ping通, 显示响应时间
      PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
      64 bytes from 8.8.8.8: icmp_seq=1 ttl=118 time=14.3ms
```

3.2 POC 2: 分号注入执行命令

```
请求: POST /ping ip=127.0.0.1;id
命令: ping -c 3 127.0.0.1;id
      ^^
      注入点: 分号分隔两条命令
结果: ping 127.0.0.1 之后再执行 id 命令
      输出 uid=0(root) gid=0(root) groups=0(root)
      → 证明命令注入成功! 当前为root用户
```

3.3 POC 3: 管道符执行任意命令

```
请求: POST /ping ip=127.0.0.1|whoami
命令: ping -c 3 127.0.0.1|whoami
      ^
      注入点: 管道符将输出重定向
结果: root
      → 只输出 whoami 的结果, ping输出被管道丢弃
```

3.4 POC 4: 读取系统文件

```
请求: POST /ping ip=127.0.0.1;cat /etc/hostname
命令: ping -c 3 127.0.0.1;cat /etc/hostname
结果: kali
      → 成功读取服务器主机名
```

四、Shell元字符攻击大全

元字符	名称	示例	效果
;	分号	ping 8.8.8.8;id	顺序执行多条命令
	管道	ping 8.8.8.8 id	将左侧输出作为右侧输入
	逻辑或	ping x id	左侧失败时执行右侧
&&	逻辑与	ping 8.8.8.8&&id	左侧成功时执行右侧
`	反引号	ping `id`	先执行反引号内的命令
\$()	命令替换	ping \$(id)	先执行括号内的命令
>	重定向	echo a > /tmp/1	写入文件
&	后台运行	ping 8.8.8.8&id	左侧后台运行，右侧前台执行

五、修复方案

5.1 避免 **shell=True**

```
# 修复前（有漏洞）
command = f"ping -c 3 {ip}"
subprocess.check_output(command, shell=True)

# 修复后（安全）
subprocess.check_output(
    ["ping", "-c", "3", ip],
    shell=False # 不使用shell，参数作为列表传入
)
```

5.2 输入白名单校验

```
import ipaddress

def validate_ip(ip_str):
    # 只允许合法的IP地址格式
    try:
        ipaddress.IPv4Address(ip_str)
        return True
    except ipaddress.AddressValueError:
        return False
```

5.3 特殊字符过滤

```
import re
# 只允许IP地址中的字符
if not re.match(r"^[0-9.]+$", ip):
    return "非法输入"
```

六、修复后验证测试

测试	请求	修复前	修复后
正常Ping	ip=8.8.8.8	正常执行	[OK] 正常
分号注入	ip=127.0.0.1;id	执行id命令	[OK] 注入失效
管道注入	ip=127.0.0.1 whoami	执行whoami	[OK] 注入失效
读文件	ip=;cat /etc/passwd	读取成功	[OK] 注入失效
非法IP	ip=abc	ping报错	[OK] 格式校验

服务端日志记录

[命令注入漏洞] 用户 admin 执行命令: ping -c 3 8.8.8.8
[命令注入漏洞] 命令执行成功: ping -c 3 8.8.8.8

[命令注入漏洞] 用户 admin 执行命令: ping -c 3 127.0.0.1;id
[命令注入漏洞] 命令执行成功: ping -c 3 127.0.0.1;id

[命令注入漏洞] 用户 admin 执行命令: ping -c 3 127.0.0.1|whoami
[命令注入漏洞] 命令执行成功: ping -c 3 127.0.0.1|whoami

[命令注入漏洞] 用户 admin 执行命令: ping -c 3 127.0.0.1;cat /etc/hostname
[命令注入漏洞] 命令执行成功: ping -c 3 127.0.0.1;cat /etc/hostname

七、总结

7.1 漏洞归纳

漏洞类型	OWASP Top 10	根因	攻击难度	危害等级
命令注入	A03:2021 注入	shell=True + f-string拼接	极低	严重

7.2 安全开发原则

- 1. 永远避免 shell=True，使用参数列表形式执行命令
- 2. 对用户输入做严格的白名单校验（IP地址用 ipaddress 库验证）
- 3. 过滤特殊字符：; | & ` \$ () {} < > !
- 4. 使用 shlex.quote() 对参数进行转义
- 5. 遵循最小权限原则，Web服务不要以root用户运行

7.3 学习心得

命令注入漏洞的利用极其简单，只需要在输入框中输入 ;id 就能直接执行系统命令。这次的实验让我深刻理解了为什么安全开发规范中反复强调"永远不要用shell=True"和"永远不要拼接命令"。一个简单的 ping 功能，如果没有安全意识到处都是突破口。

— 报告完 —